File Copy Implementation

1. Team Members and Responsibility

As this is an individual assignment, all tasks were performed by a single member:

- **Kuan, Chin-Wei (官京緯)**: 隨班附讀 (ID: 114054811)

2. Project Overview

This project is a low-level file utility implemented in **C** using **POSIX system calls**. It demonstrates the direct interaction between user-level applications and the operating system kernel for file I/O operations.

File Copy Logic

The program operates through the following steps:

- **User Prompting**: Requests the source and destination filenames from the user.
- **File Descriptors**: Opens the source file in read-only mode and the destination file with creation/truncation flags.
- **Buffered Transfer**: Performs a byte-by-byte copy using a 1024-byte buffer to optimize read/write performance.

3. Features

- **Direct System Calls**: Utilizes `open()`, `read()`, `write()`, and `close()` for low-level I/O.
- **Robust Error Handling**: Checks for file existence, directory permissions, and potential write errors.
- **POSIX Compliant**: Developed and tested on macOS (Darwin), ensuring compatibility with most Unix-like systems.

4. Compilation and Configuration Instructions

Environment

- **OS**: Developed on **macOS (Apple Silicon)**; compatible with Unix-like systems (Linux/BSD).
- **Requirements**: `gcc` or `clang` compiler.

Build the Project

Enter the project folder and use one of the following command to compile:

Method A. Using Makefile(Recommended):

```
make
```

Method B. Direclty command to complie

```
gcc FileCopy.c -o FileCopy
```

Run the Program

Execute the binary and follow the on-screen prompts:

```
./FileCopy
# Enter source file name: source.txt
# Enter destination file name: dest.txt
```

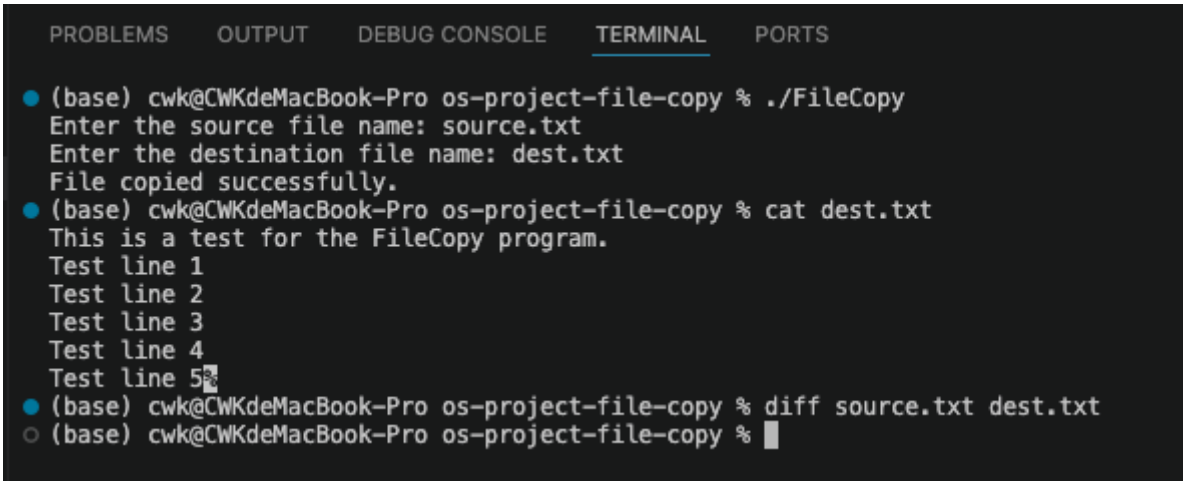
Verification

To ensure the copy is identical to the source, use the diff command:

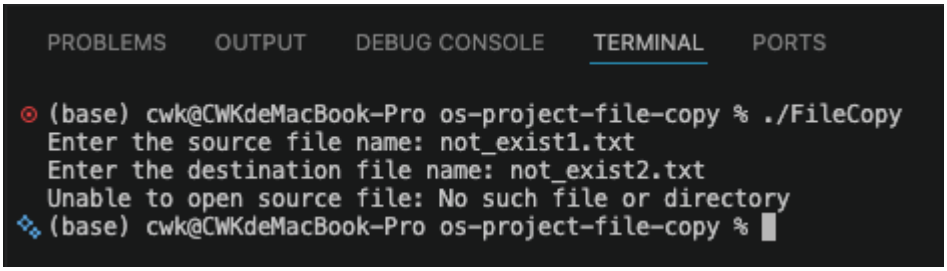
```
diff source.txt dest.txt
# No output means the files are identical.
```

5. Test Results (System Trace & Error Handling)

a. Basic Feature: Successful Copy



b. Error Handling: Source File Not Found



c. Error Handling: Permission Denied

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(base) cwk@CWKdeMacBook-Pro os-project-file-copy % ./FileCopy
Enter the source file name: secret.txt
Enter the destination file name: dest.txt
Unable to open source file: Permission denied
(base) cwk@CWKdeMacBook-Pro os-project-file-copy %
```

d. System Trace: dtruss via SIP

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(base) cwk@CWKdeMacBook-Pro os-project-file-copy % sudo dtruss ./FileCopy
dtrace: system integrity protection is on, some features will not be available

SYSCALL(args)             = return
Enter the source file name: source.txt
Enter the destination file name: dest.txt
File copied successfully.
```

The `dtruss` output is restricted by macOS **System Integrity Protection (SIP)** on modern macOS for security reasons. Attempting to bypass this by disabling SIP is not recommended for the potentional risks and unpredictable system instability.

However, the program's logic is verified via diff and binary comparison. To view traces, ensure the binary is not restricted by SIP or run in a sandbox-relaxed environment.

6. File Structure

- `FileCopy.c`: Core source code (POSIX I/O logic).
- `Makefile`: Automation script for compilation and cleanup.
- `README.md`: Project documentation.
- `snapshot`: Directory containing execution screenshots and test results.

Author: Kuan, Chin-Wei (官京緯)

Date: March 30, 2026